TurboQuant on Quantized Models:
Solving Compound Quantization Error with Pre-RoPE Compression and KV Compaction

Onur Gökyıldız
BHI Research, Beyond Horizons Industries
Istanbul, Turkey
ogokyildiz@beyondhorizonsindustries.com

April 2026

Abstract

Every TurboQuant implementation we tested produces catastrophic output on GGUF quantized models. The reason is compound quantization error: weight quantization noise gets amplified through a second round of KV cache quantization, and softmax makes it worse. On Qwen 2.5 7B Q4_K_M, a representative Python implementation produces perplexity of 3,556 (baseline: 5.18). That is not a typo.

This paper presents tq-kv, a Rust implementation of TurboQuant (ICLR 2026) that actually works on quantized models. We solve compound error through the 3-Fix Framework (sink token preservation, past-only quantization, cache state management) and introduce Pre-RoPE key quantization, which compresses keys before rotary position embedding to exploit position-independent channel statistics. Pre-RoPE reduces the perplexity gap by 34-59% at identical compression ratio. We also integrate KV Compaction, an attention-matching token reduction method orthogonal to bit compression, achieving combined compression up to 100-400x.

On the systems side, we replace the dense QJL projection from the original paper with a structured Hadamard variant (SRHT-QJL) that is 115x faster and produces +4.5 dB better SNR. Fused attention scoring eliminates key decompression via AVX2+FMA centroid lookup, yielding 6-8.9x speedup. An O(1) incremental cache architecture cuts per-token overhead from 608 ms to 0.65 ms.

We evaluate across five architectures (Llama-3, Qwen2.5, Gemma, Mistral, Phi-3) on both FP16 and GGUF Q4_K_M models. At 4-bit, tq-kv achieves +3.7% PPL delta with Pre-RoPE on GGUF (vs +17% for standard TQ and 3,556 for existing implementations), 9/9 needle-in-a-haystack pass rate, and 7.5-14.2x key compression. The system is 13.7K lines of Rust, ships as a crate on crates.io, and includes a complete inference engine with OpenAI-compatible API.

Keywords: *KV cache compression, compound quantization error, Pre-RoPE quantization, KV compaction, TurboQuant, Rust, GGUF, Lloyd-Max codebook, Walsh-Hadamard transform, SIMD*

1. Introduction

The memory cost of LLM inference is dominated by the key-value cache. For Llama-3 8B at 4096 tokens, the keys alone consume 256 MB. For Qwen2.5 72B, 640 MB. At 32K context with Llama 70B, the KV cache eats 20 GB. This is not a theoretical concern; it directly determines how many concurrent users a GPU can serve, and at what context length.

TurboQuant (Ashkboos, Zandieh et al., ICLR 2026) offers an appealing solution. The algorithm applies a randomized Hadamard transform to decorrelate outlier-heavy key vectors, then quantizes the resulting near-Gaussian coordinates using Lloyd-Max optimal codebooks. It is provably near-optimal in distortion rate, requires no training or calibration data, and operates on one vector at a time. On paper, this is exactly what KV cache compression needs.

In practice, it does not work on the models people actually run.

Nearly all local LLM deployment uses GGUF-quantized models (Q4_K_M, Q5_K_M). When you apply TurboQuant's KV cache compression on top of already-quantized weights, the errors compound multiplicatively through softmax. The result is not a graceful quality degradation. It is five languages mixed together, then gibberish, within 50 tokens. We tested every publicly available TurboQuant implementation on Qwen 2.5 7B Q4_K_M. The best produced PPL of 3,556 against a baseline of 5.18. Most crashed or produced empty output.

This paper describes tq-kv, a Rust implementation that makes TurboQuant work on quantized models. Our contributions fall into three categories.

First, the compound error problem. We introduce the 3-Fix Framework: preserve attention sink tokens in FP16, protect the current token via past-only quantization, and manage cache state across conversations. These three fixes, each simple individually, are the difference between PPL 3,556 and PPL 6.07.

Second, Pre-RoPE key quantization. Standard TurboQuant compresses keys after rotary position embedding has been applied. Post-RoPE keys have position-dependent per-channel statistics, which degrades the Gaussian assumption that Lloyd-Max codebooks rely on. We compress before RoPE. The statistics are position-independent, the codebook fit is better, and the PPL gap shrinks by 34-59% at no extra compression cost.

Third, systems engineering. We replace dense QJL with a structured Hadamard variant that is 115x faster. Fused attention computes scores directly from compressed indices without decompression. An append-only cache architecture eliminates the O(N) recompression that naive implementations require. KV Compaction reduces token count orthogonally to bit compression, enabling combined compression ratios exceeding 100x. The entire system is 13.7K lines of Rust with 86 passing tests, ships on crates.io, and includes an inference engine with model hub, calibration pipeline, and OpenAI-compatible API server.

2. Related work

2.1 Token eviction

H2O identifies high-attention "heavy hitter" tokens and evicts the rest. StreamingLLM preserves the first few tokens (attention sinks) and a sliding window of recent ones. Both are lightweight but permanently discard information. They fail on tasks that require retrieval from arbitrary context positions.

2.2 KV cache quantization

KIVI (ICML 2024) applies 2-bit asymmetric quantization, quantizing keys per-channel and values per-token. KVQuant (NeurIPS 2024) extends this to 10M-token contexts with per-channel pre-RoPE key quantization, non-uniform codebooks, and dense-and-sparse outlier handling. GEAR uses low-rank approximation followed by uniform quantization. All operate at the integer level and do not exploit geometric structure of the KV vectors.

2.3 Rotation-based methods

QuaRot applies randomized Hadamard transforms to eliminate outliers before 4-bit weight quantization. QuIP# uses Hadamard incoherence with lattice codebooks for extreme weight compression. Both demonstrate that orthogonal rotations improve quantization quality by distributing information more uniformly across coordinates. TurboQuant applies this insight specifically to the KV cache, pairing rotation with Lloyd-Max optimal scalar quantization and QJL residual correction.

2.4 The implementation landscape

Following TurboQuant's publication, several independent implementations appeared. None works on GGUF models. Table 1 compares the existing landscape.

Feature	tq-kv (ours)	turboquant_plus	turboquant (PyPI)	Triton impl.
Language	Rust	Python/C	Rust	Python/CUDA
GGUF Q4_K_M PPL	6.07 (+17%)	3,556 (broken)	N/A	N/A
Pre-RoPE	Yes (5.40)	No	No	No
KV Compaction	Yes (~25x)	No	No	No
CUDA GPU	Yes	No	No	Yes
AVX2 SIMD fused attn	Yes (8.9x)	No	No	No
C FFI / llama.cpp	Yes	Yes (fork)	No	No
Incremental O(1) cache	Yes	No	No	No
Tests passing	86	N/A	N/A	N/A

Table 1: TurboQuant implementations compared (March 2026).

3. Background

3.1 The TurboQuant algorithm

TurboQuant compresses each key vector k through three stages. First, a randomized Hadamard transform R = H_d * D / sqrt(d) rotates the vector so that its coordinates become approximately independent and Gaussian. Since R is orthogonal, inner products are preserved exactly: q^T k = (Rq)^T(Rk). The transform runs in O(d log d) via the Fast Walsh-Hadamard algorithm.

Second, each coordinate of the rotated vector is quantized using a Lloyd-Max optimal scalar codebook for the Gaussian distribution. At b bits per coordinate, this produces 2^b centroids that minimize mean-squared error for a known source distribution. The quantized representation is the centroid indices (b bits each) plus a single norm/scale factor per vector.

Third, an optional QJL (Quantized Johnson-Lindenstrauss) step projects the quantization residual through a random matrix and stores only sign bits. This theoretically provides unbiased inner product estimation at 1 additional bit per dimension.

3.2 Why GGUF breaks everything

The original paper evaluates on FP16 models. In the local inference ecosystem, nobody runs FP16. The standard format is GGUF with Q4_K_M or Q5_K_M weight quantization, which fits 70B models on consumer hardware.

The problem is that quantized weight matrices produce key vectors that already contain quantization noise. When TurboQuant applies a second round of quantization to these noisy keys, the errors do not simply add. Softmax exponentiates the attention logits, which means even small errors in key-query dot products get amplified nonlinearly. The attention distribution can shift dramatically, and the model's output degenerates.

This is not a subtle effect. On Qwen 2.5 7B Q4_K_M, without any fixes, TurboQuant produces language mixing (5+ languages in a single response) within 50 tokens. With the fixes described in Section 5.1, we get PPL of 6.07 against a baseline of 5.18.

4. System architecture

tq-kv is a two-crate Rust workspace: the tq-kv library (6,783 LOC, published on crates.io) and the tq-engine binary (6,905 LOC). Total: 13.7K LOC across 32 source files with 86 tests.

4.1 Compression pipeline

The pipeline processes each key vector through the following stages: optional Pre-RoPE capture (compress before position embedding), optional pre-rotation centering (subtract per-channel bias from calibration), randomized Hadamard transform (O(d log d)), per-group Lloyd-Max quantization with adaptive sigma = group_norm / sqrt(group_size), norm correction (stored_norm = norm^2 / recon_norm), and optionally residual quantization, outlier preservation, or SRHT-QJL error correction. Output is bit-packed indices plus per-group norm scalars.

4.2 Five-segment attention

During inference, attention operates over five distinct segments, each with different precision:

- Sink tokens (first N): full FP16, always lossless. These are the attention anchors.
- Cold decayed: older tokens auto-demoted to lower bit width via index remapping.
- Compacted segment: synthetic KV pairs with attention biases (beta), produced by KV Compaction.
- Hot compressed: recently added tokens at the configured TurboQuant bit width.
- Current token: FP16 original (past-only quantization). Compressed into cache for future use.

This segmented design allows each region to use the precision it actually needs. Sink tokens and the current token are always lossless. Old tokens get demoted because their attention contribution diminishes. Compacted tokens are a compressed summary of the middle region.

4.3 Incremental cache

Each new key is independently compressed and appended as packed indices plus a norm scalar. This is an O(1) operation. The naive alternative (decompress everything, append, recompress) is O(N) and takes about 608 ms at 4096 context. Our approach takes 0.65 ms. That is a 935x difference.

4.4 The tq-engine product

The library powers tq-engine, a complete inference engine with model hub (tq pull, tq list), OpenAI-compatible HTTP API with SSE streaming (tq serve), interactive terminal chat (tq chat), automated perplexity evaluation (tq perplexity), calibration from real activations (tq calibrate), and ablation sweeps (tq ablate). It auto-detects five GGUF architectures: Qwen2, Llama, Mistral, Phi-3, and Gemma 2.

5. Technical contributions

5.1 The 3-Fix framework

The most practically important thing in this paper. Without it, TurboQuant on GGUF is useless.

Fix 1: Sink token preservation

The first S tokens (default S=4) stay in FP16 within the compressed cache. StreamingLLM showed that initial tokens receive disproportionate attention regardless of content. Quantizing these tokens corrupts the attention distribution more than quantizing any other position. By keeping them at full precision, we reduce attention distribution error by 81%. On its own, this fix brings PPL from the catastrophic range down to +34.3% over baseline.

Fix 2: Past-only quantization (POQ)

The current token's key participates in attention at FP16 precision, then gets compressed into the cache for future tokens. This protects the highest-impact position (the one with attention weight of 1.0 in causal self-attention of the query). Combined with Fix 1, coherent generation extends to 200+ tokens.

Fix 3: Cache state management

Hard reset of the compressed KV state between conversations. Without this, stale RoPE-mismatched keys from previous conversations contaminate the cache. With all three fixes, generation is stable indefinitely.

Configuration	GGUF PPL	Status
No TurboQuant (baseline)	5.18	Working
turboquant_plus (symmetric)	3,556	Catastrophic
tq-kv with 3-Fix	6.07	Working (+17%)
tq-kv with Pre-RoPE	5.40	Best (+5.6%)

Table 2: Compound quantization error on Qwen 2.5 7B Q4_K_M.

5.2 Pre-RoPE key quantization

This is our biggest quality improvement. Standard TurboQuant compresses keys after rotary position embedding has been applied. RoPE rotates pairs of coordinates by position-dependent angles, which means each token position creates different per-channel statistics. The Lloyd-Max codebook is computed for a single Gaussian distribution, so position-dependent variation degrades the fit.

We compress keys before RoPE instead. Pre-RoPE keys have consistent, position-independent per-channel statistics that match the Gaussian assumption much better. At decode time, we decompress and apply RoPE dynamically per stored position. This adds a decompress-then-RoPE step (so fused attention is incompatible, with automatic fallback), but the quality improvement is substantial.

Tokens	Baseline	Standard TQ (delta)	Pre-RoPE (delta)	Gap reduction
475	5.117	5.820 (+13.7%)	5.403 (+5.6%)	59%
793	4.901	5.250 (+7.1%)	5.125 (+4.6%)	35%
2106	1.823	1.925 (+5.6%)	1.890 (+3.7%)	34%

Table 3: Pre-RoPE vs standard TQ on Qwen 2.5 7B Q4_K_M, modern English.

The gap reduction ranges from 34% to 59% depending on context length. At 475 tokens, Pre-RoPE cuts the delta from +13.7% to +5.6%. The idea comes from KVQuant's observation about pre-RoPE channel statistics, but to our knowledge this is the first implementation of Pre-RoPE within the TurboQuant framework.

5.3 KV compaction

Bit compression and token reduction are orthogonal. TurboQuant reduces bits per coordinate. KV Compaction (following Zweiger et al., 2026) reduces the number of tokens. Combined, they multiply: 8x bit reduction times 20x token reduction is 160x.

The compaction algorithm selects the top-t keys by mean attention weight across all GQA-mapped query heads, fits per-key biases (beta) via non-negative least squares to preserve the softmax partition function, and fits synthetic values via ridge regression with adaptive regularization. During attention, compacted keys receive their beta bias added to the logits, and synthetic values are used in the V multiply.

Config	PPL	Delta	Est. compression
Baseline (no TQ)	1.823	--	1x
Standard TQ 4-bit	1.925	+5.6%	~7.5x
Pre-RoPE 4-bit	1.890	+3.7%	~7.5x
TQ + Compact (500t/30%)	2.227	+22.2%	~25x
Pre-RoPE + Compact	2.281	+25.1%	~25x
Full stack (Pre-RoPE+V4+Compact)	2.84	+55.8%	~100x

Table 4: KV Compaction results on Qwen 2.5 7B Q4_K_M, 2106 tokens.

5.4 SRHT-QJL

The original paper specifies a dense random Rademacher projection for QJL residual correction. This is O(d^2) in both time and space. We replace it with a Structured Random Hadamard Transform that is O(d log d) in time and O(d) in space.

The result is 115x faster compression and +4.5 dB better SNR than the dense version. The improvement happens because structured projections have lower variance than purely random Rademacher matrices. Five independent groups have confirmed that dense QJL hurts practical quality more than it helps. We set QJL to off by default. An adaptive mode auto-enables it at long context.

Metric	Dense QJL (paper)	SRHT-QJL (ours)	No QJL
SNR (dB)	21.5	24.8	20.3
Cosine similarity	0.9966	0.9984	0.9956
Compress overhead	29x	1.45x	1.0x
Decompress overhead	128x	1.7x	1.0x
Attention KL divergence	Baseline	2.9x lower	Baseline

Table 5: SRHT-QJL vs dense QJL (32K vectors, dim=128, 4-bit).

5.5 Fused attention scoring

Standard compressed-key attention requires decompressing all keys to FP32 before computing dot products. We skip decompression entirely. Since the Hadamard rotation R is orthogonal: dot(q, k) = dot(Rq, centroids[idx]) * sigma. Pre-rotate the query once, then each cached key costs a centroid table lookup and a scalar multiply.

The AVX2+FMA implementation processes 8 float32 values per cycle using gather instructions for centroid lookup and fused multiply-add for accumulation. Score divergence from the decompress path is 0.000001, which is numerically equivalent. Speedup is 6x at 512 keys, reaching 8.9x with larger batches. Fused attention is incompatible with Pre-RoPE (because RoPE is position-dependent and cannot be absorbed into the centroid table), so the system falls back automatically.

5.6 Additional techniques

Adaptive per-vector sigma. The paper uses fixed sigma = 1/sqrt(d). We compute sigma_i = ||k_i|| / sqrt(d) per vector, matching the actual post-rotation variance. Confirmed by kurtosis measurement: 35.3 before rotation, 3.3 after (Gaussian ideal is 3.0).

K/V asymmetric compression. Keys go through TurboQuant. Values use simpler absmax quantization at 8-bit (+0.2% extra PPL, 2x savings) or 4-bit (+1.3% extra PPL, 3.2x savings). The asymmetry is justified because softmax amplifies key errors exponentially while value errors scale linearly.

Temporal decay. Old tokens are automatically demoted to lower bit width via centroid index remapping. No decompression needed. Over 30% extra savings at long context.

Per-head adaptive bitwidth. Sparse attention heads (single token dominant) get 2-bit. Diffuse heads get 4-bit. Calibrated automatically from key norm variance.

Calibration pipeline. The tq calibrate command collects real key activations and computes: per-channel bias (pre-rotation centering), SmoothQuant-style channel scales, PCA rotation matrix via Jacobi eigendecomposition, calibrated codebooks from real distributions, and per-head importance for adaptive bitwidth. Results are saved as JSON and auto-loaded at startup.

C FFI. Opaque-handle API (tq_init, tq_compress_and_append, tq_fused_attention, tq_layer_*) designed for llama.cpp GQA integration. Ships as libtq_kv.a with tq_kv.h header.

6. Experimental evaluation

6.1 Setup

Hardware: RTX 3080 10 GB, i9-13900K, 64 GB RAM, Windows 11, CUDA 13.2, Rust 1.91 release mode. Perplexity measured via automated tq perplexity and tq ablate commands. Unless noted, models use GGUF Q4_K_M format with skip=4 (first 4 layers uncompressed) and sink=4 (first 4 tokens FP16).

6.2 GGUF compound error

Config	PPL	Delta	Notes
Baseline (no TQ)	5.178	--	
K4-bit, V-fp16	6.065	+17.1%	Standard recommended
K4-bit, V-8bit	6.076	+17.3%	V8 nearly free (+0.2%)
K4-bit, V-4bit	6.143	+18.6%	V4 acceptable
Pre-RoPE K4-bit, V-fp16	5.40	+5.6%	Best quality

Table 6: Qwen 2.5 7B Q4_K_M perplexity on wikitext-2.

6.3 FP16 models

Bits	PPL	Delta	Compression
Baseline	10.740	--	1.0x
4-bit	11.967	+11.4%	7.5x
3-bit	13.933	+29.7%	9.8x
2-bit	27.696	+157.9%	14.2x

Table 7: Qwen 2.5 0.5B FP16 (safetensors), wikitext-2.

6.4 Compression quality

Model	Bits	Ratio	SNR (dB)	Cosine sim
Llama-3 8B	2	14.2x	9.4	0.943
Llama-3 8B	3	9.8x	14.7	0.984
Llama-3 8B	4	7.5x	20.4	0.996
Gemma 3 4B	2	15.1x	9.3	0.942
Gemma 3 4B	3	10.2x	14.7	0.984

Table 8: Per-layer compression quality.

6.5 NIAH

Needle-in-a-haystack test: inject "The secret code for the treasure vault is GOLDEN-PHOENIX-7742" at varying depths in ~4K tokens. The model must retrieve the code through compressed KV cache. Result: 9/9 pass at both 4-bit and 2-bit, across depths 10%, 25%, 50%, 75%, and 90%. Compressed keys preserve enough angular resolution for precise information retrieval.

6.6 VRAM savings

Model	Context	FP16 KV	TQ 4-bit	TQ 2-bit	Savings
Qwen 2.5 7B	4K	256 MB	34 MB	18 MB	7.5-14.2x
Qwen 2.5 72B	4K	640 MB	85 MB	45 MB	7.5-14.2x
Llama 3.1 70B	32K	20 GB	2.7 GB	1.4 GB	7.5-14.2x

Table 9: VRAM projection (keys only). With compaction, effective compression reaches 100-400x.

6.7 Throughput

Model	CUDA tok/s	CPU tok/s	GPU speedup	TTFT
Qwen 2.5 7B	28.2	6.04	4.7x	0.118s
Llama 3.1 8B	19.2	3.47	5.5x	0.126s

Table 10: CUDA vs CPU throughput (TQ 4-bit, RTX 3080).

TurboQuant adds 8-17% CPU throughput overhead, decreasing with longer generation as compression cost is amortized. The 2.8x gap versus llama.cpp on CPU decode is candle vs GGML framework overhead, not TQ-specific. On CUDA, tq-engine approaches llama.cpp CPU performance while delivering 7.5-14.2x memory savings.

7. Discussion

7.1 The compound error problem matters more than compression quality

Most KV cache compression papers evaluate on FP16 models. In the real world, people run quantized models. The gap between "works on FP16" and "works on Q4_K_M" is the gap between a research prototype and a useful tool. Our 3-Fix Framework is not algorithmically sophisticated. Sink token preservation, POQ, and cache reset are each straightforward ideas. But without them, the algorithm is broken on the workloads that matter.

7.2 Pre-RoPE: a 34-59% quality improvement for free

Pre-RoPE quantization does not change the compression ratio. It does not add parameters or calibration data. It just compresses keys at a point in the pipeline where their statistics are better behaved. The cost is that fused attention becomes incompatible (you need to apply RoPE after decompression), so the system falls back to the decompress path. In practice, this trade-off is almost always worth it.

7.3 QJL: theoretically elegant, practically marginal

Five independent groups tested QJL across different codebases and arrived at the same conclusion: off by default. The variance QJL introduces outweighs its bias correction in nearly all practical scenarios. Our SRHT variant improves the situation (+4.5 dB SNR vs dense) but the benefit still does not justify the 1.45x compression overhead for most workloads. The adaptive mode (enable at long context) is the right default.

7.4 Why Rust

The choice was deliberate. Rust's ownership model prevents the memory bugs that plague C/C++ inference engines. Its zero-cost abstractions and SIMD intrinsics deliver competitive performance. The no_std core enables embedded deployment. And Rayon's fearless concurrency maps naturally to parallel multi-head processing. The practical impact: 86 tests, all memory-safe, zero undefined behavior, with C FFI interoperability for the existing ecosystem.

7.5 Compaction optimization: multi-query reference matters

Our initial compaction implementation used a single query to score key importance. The results were poor. Switching to all GQA-mapped query heads as reference and mean-based scoring (instead of max) made a dramatic difference.

Config	Before optimization	After optimization	Improvement
TQ + Compact 500t/30%	5.777	2.227	2.6x
TQ + Compact 1000t/20%	2.982	2.367	1.26x

Table 11: Impact of multi-query reference optimization on compaction quality.

The lesson is that compaction quality is extremely sensitive to the key selection criterion. Using all available queries provides a much more robust importance signal than any single query.

7.6 Value compression is nearly free

One of the more surprising findings is how cheaply values can be compressed. Adding 8-bit value quantization on top of 4-bit key compression costs only +0.2% extra PPL while halving the value cache size. Even 4-bit values add only +1.3%.

The reason comes down to how attention works. Keys determine the attention weights through softmax, which is an exponential function. Small errors in key-query dot products get amplified exponentially. Values, by contrast, are linearly combined with weights that are already normalized by softmax. Linear combination is far more forgiving of quantization noise than exponentiation. This means the optimal bit budget allocation is heavily asymmetric: spend your bits on keys.

Value config	Extra PPL vs K-only	Value savings
V-fp16 (default)	--	1.0x
V-8bit (TQ_VBITS=8)	+0.2%	2.0x
V-4bit (TQ_VBITS=4)	+1.3%	3.2x

Table 12: Value compression cost on Qwen 2.5 7B Q4_K_M.

7.7 Context length scaling

A production system cannot afford quality degradation that worsens with context length. We verified that 4-bit TurboQuant maintains a stable PPL delta across context lengths from 256 to 2048 tokens on Trendyol Llama-3 8B.

Context length	Standard PPL	4-bit PPL	Delta
256 tokens	13.863	14.076	+1.5%
512 tokens	11.368	11.585	+1.9%
1024 tokens	7.441	7.474	+0.4%
2048 tokens	9.651	9.718	+0.7%

Table 13: PPL vs context length (Trendyol Llama-3 8B, wikitext-2, 4-bit).

The delta stays between +0.4% and +1.9%. There is no quality cliff. This stability comes from the per-vector adaptive sigma computation, which correctly scales quantization boundaries for each vector regardless of its position in the sequence.

7.8 Attention KL divergence across context

We measured the KL divergence between attention distributions computed from compressed versus original keys at varying context lengths. SRHT-QJL consistently reduces KL divergence by 2.8-2.9x compared to no QJL, and this improvement holds constant across context lengths from 64 to 16,384 tokens.

Context	KL (no QJL)	KL (SRHT-QJL)	Reduction
64	0.000270	0.000093	2.9x
256	0.000283	0.000099	2.9x
1,024	0.000289	0.000103	2.8x
4,096	0.000288	0.000102	2.8x
16,384	0.000288	0.000103	2.8x

Table 14: Attention KL divergence vs context length (4-bit, dim=128).

The flat scaling is notable. It means the quality of compressed attention does not degrade as the cache grows, which is the critical property for long-context deployment.

7.9 Fused attention: detailed breakdown

Bits	Fused (us)	Decompress+dot (us)	Speedup	Score divergence
2-bit	68	411	6.0x	0.000001
3-bit	97	584	6.0x	0.000001
4-bit	83	503	6.1x	0.000001

Table 15: Fused vs decompress-then-compute attention (Llama-3 8B, 512 cached keys).

Score divergence of 0.000001 across all bit widths confirms numerical equivalence. The speedup comes from eliminating the decompression step entirely. For each cached key, the fused path does a centroid lookup (b-bit index to f32 centroid) and a scalar multiply with the stored norm. On AVX2, gather instructions load 8 centroids per cycle and FMA computes the accumulated dot product.

With larger batch sizes and longer cached sequences, the speedup increases to 8.9x because the overhead of pre-rotating the query (O(d log d), done once) is amortized over more key lookups.

7.10 Incremental cache: why O(1) matters

The difference between O(1) and O(N) per-token cache cost becomes dramatic at long context. At 4096 tokens with 2-bit compression:

Architecture	Per-token cache cost	Scaling
Naive (decompress + recompress all)	~608 ms	O(N)
tq-kv (compress 1 + append)	~0.65 ms	O(1)
Improvement	935x	

Table 16: Per-token cache overhead comparison.

At 32K context, the naive approach would take nearly 5 seconds per token just for cache management. Our approach stays at 0.65 ms regardless of context length. This is not an optimization; it is a prerequisite for any system that claims to support long context.

7.11 The Gaussianity assumption

The entire Lloyd-Max codebook approach relies on post-rotation coordinates being approximately Gaussian. We verified this empirically.

Metric	Before Hadamard	After Hadamard	Gaussian ideal
Kurtosis	35.3	3.3	3.0

Table 17: Gaussianity verification of key vectors before and after Hadamard rotation.

Before rotation, key vectors exhibit kurtosis of 35.3, which means heavy tails and outliers that would destroy any fixed codebook. After the Hadamard transform, kurtosis drops to 3.3, close to the Gaussian ideal of 3.0. The Lloyd-Max centroids, which were pre-computed assuming a Gaussian source, are therefore near-optimal for the actual post-rotation distribution.

7.12 Generation throughput: detailed analysis

Model	Tokens	Standard tok/s	TQ 4-bit tok/s	Delta	TTFT std	TTFT TQ
Qwen 2.5 7B	50	5.66	4.75	-16%	2.31s	2.51s
Qwen 2.5 7B	200	6.55	6.04	-8%	2.22s	2.32s
Llama 3.1 8B	200	4.20	3.47	-17%	2.69s	2.75s

Table 18: CPU generation throughput (i9-13900K, Q4_K_M).

The overhead pattern is telling: at 50 tokens, TQ costs 16% throughput. At 200 tokens, only 8%. Compression is a fixed cost per token, while the generation benefit (reduced memory bandwidth for cached keys) grows with sequence length. At sufficiently long context, TQ should break even or become faster than uncompressed inference. We have not yet demonstrated this crossover point empirically, but the trend is clear.

Comparing against llama.cpp on the same hardware: llama.cpp decodes Qwen 2.5 7B at 16.7 tok/s versus tq-engine's 5.8 tok/s. This 2.8x gap comes from the Candle framework overhead relative to GGML's hand-optimized kernels. TQ's contribution to the gap is only 2-11% depending on generation length. On CUDA, tq-engine reaches 28.2 tok/s (Qwen) and 19.2 tok/s (Llama), which approaches llama.cpp CPU throughput while simultaneously compressing the KV cache by 7.5-14.2x.

8. Limitations and future work

The CPU decode throughput gap versus llama.cpp (about 2.8x) is a real limitation, even though it comes from the Candle framework rather than TurboQuant. Users comparing end-to-end performance will see it regardless. CUDA helps (4.7-5.5x over CPU) but required custom RmsNorm, RoPE, and softmax kernels because Candle's quantized_nn module lacks CUDA support.

The 3-Fix Framework has been validated primarily on Qwen 2.5 7B Q4_K_M. Broader validation across other quantization formats and model families is needed. The Pre-RoPE quality improvement was measured on modern English text; multilingual and code generation evaluation would strengthen the claims.

KV Compaction at aggressive ratios (20-50x token reduction) adds meaningful quality degradation (+22-30% PPL). Finding the right threshold for a given quality budget requires per-model tuning.

Open directions include: calibrated codebooks from real activations (the infrastructure exists, large-scale evaluation does not), learned rotation matrices (SpinQuant-style, replacing random Hadamard), Flash Attention tiling for fused TQ attention at very long context, Metal shaders for Apple Silicon, and PyO3 Python bindings for the HuggingFace ecosystem.

9. Conclusion

We presented tq-kv, a Rust implementation of TurboQuant that works on GGUF quantized models where every other implementation fails. The 3-Fix Framework solves compound quantization error through three simple mechanisms. Pre-RoPE key quantization reduces the quality gap by 34-59% by compressing keys before position embedding. KV Compaction adds orthogonal token reduction for combined compression exceeding 100x.

On the systems side, SRHT-QJL is 115x faster than the paper's dense QJL with better quality. Fused attention eliminates decompression for 6-8.9x speedup. O(1) incremental caching provides 935x overhead reduction. The system is 13.7K lines of Rust, 86 tests, and ships as both a library crate and a complete inference engine.

The gap between "works on FP16" and "works on Q4_K_M" turned out to be the most important engineering challenge in this entire project. Closing it required no mathematical breakthroughs, just careful observation of what actually breaks and systematic fixes. We hope this work is useful to others building KV cache compression for real-world deployment.

References

[1] Ashkboos, S., Zandieh, A., Daliri, M., Hadian, M., Mirrokni, V. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate. ICLR 2026. arXiv:2504.19874.

[2] Liu, Z., Yuan, J., et al. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. ICML 2024. arXiv:2402.02750.

[3] Hooper, C., et al. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. NeurIPS 2024. arXiv:2401.18079.

[4] Ashkboos, S., et al. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. NeurIPS 2024.

[5] Tseng, A., et al. QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. ICML 2024. arXiv:2402.04396.

[6] Zandieh, A., et al. QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead. ICML 2024.

[7] Daliri, M., et al. PolarQuant: Quantization with Polar Coordinates. AISTATS 2026.

[8] Zweiger, A., et al. Attention-Matching KV Cache Compaction. 2026.

[9] Zhang, Z., et al. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. NeurIPS 2023.

[10] Xiao, G., et al. Efficient Streaming Language Models with Attention Sinks. ICLR 2024.

[11] Kang, H., et al. GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM. 2024.

[12] Lloyd, S.P. Least Squares Quantization in PCM. IEEE Trans. Information Theory, 28(2), 1982.

[13] Fino, B.J., Algazi, V.R. Unified Matrix Treatment of the Fast Walsh-Hadamard Transform. IEEE Trans. Computers, 1976.

[14] Gerganov, G. llama.cpp: LLM inference in C/C++. https://github.com/ggerganov/llama.cpp

[15] Hugging Face. Candle: Minimalist ML framework for Rust. https://github.com/huggingface/candle

[16] Dao, T. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. ICLR 2024.

[17] Kwon, W., et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023.

[18] Lin, Y., et al. QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving. 2024.

[19] Liu, J., et al. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. 2024.

[20] Sheng, Y., et al. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. ICML 2023.